

How to Take Input From User in Java?

Java brings various Streams with its I/O package that helps the user perform all the [Java input-output operations](#). These streams support all types of objects, data types, characters, files, etc. to fully execute the I/O operations. Input in Java can be with certain methods mentioned below in the article.

Methods to Take Input in Java

There are two ways by which we can take Java input from the user or from a file

- [BufferedReader Class](#)
- [Scanner Class](#)

1. Using BufferedReader Class for String Input In Java

It is a simple class that is used to read a sequence of characters. It has a simple function read that reads a character, another read which reads an array of characters, and a readLine() function which reads a line.

InputStreamReader is a class that converts the input stream of bytes into a stream of characters, allowing it to be read by BufferedReader, which expects a stream of characters.

```
import java.io.*;
class BufferedReaderDemo
{
    public static void main(String[] args) throws IOException
    {
        BufferedReader bfn = new BufferedReader(new InputStreamReader(System.in));

        System.out.println("Enter Student Name");
        String str = bfn.readLine();

        System.out.println("Enter Student PRN");
        int prn = Integer.parseInt(bfn.readLine());

        System.out.println("Entered Name is : " + str);
        System.out.println("Entered PRN is : " + prn);
    }
}
```

2. Using Scanner Class for Taking Input in Java

It is an advanced version of `BufferedReader` which was added in later versions of Java. The scanner can read formatted input. It has different functions for different types of data types.

- The scanner is much easier to read as we don't have to write throws as there is no exception thrown by it.
- It was added in later versions of Java
- It contains predefined functions to read an Integer, Character, and other data types as well.

Inbuilt Scanner functions are as follows:

- **Integer:** `nextInt()`
- **Float:** `nextFloat()`
- **String :** `next()` and `nextLine()`

```
import java.util.*;
class ScannerDemo1
{
    public static void main(String[] args)
    {

        Scanner scn = new Scanner(System.in);

        System.out.println("Enter Name");
        String str1 = scn.next();

        System.out.println("Enter PRN");
        int x = scn.nextInt();

        System.out.println("Enter Last Year Percentage");
        float f = scn.nextFloat();

        System.out.println("Entered Name is: " + str1);
        System.out.println("Entered PRN : " + x);
        System.out.println("Entered FloatValue : " + f);
    }
}
```

Differences Between `BufferedReader` and `Scanner`

- `BufferedReader` is a very basic way to read the input generally used to read the stream of characters. It gives an edge over `Scanner` as it is faster than `Scanner` because `Scanner` does lots of post-processing for parsing the input; as seen in `nextInt()`, `nextFloat()`
- `BufferedReader` is more flexible as we can specify the size of stream input to be read. (In general, it is there that `BufferedReader` reads larger input than `Scanner`)

- These two factors come into play when we are reading larger input. In general, the Scanner Class serves the input.
- BufferedReader is preferred as it is synchronized. While dealing with multiple threads it is preferred.
- For decent input, and easy readability. The Scanner is preferred over BufferedReader.